

TECH

PERFORMANCE

BROWSER / RUNTIME REACT PERFORMANCE

REACT PERFORMANCE

30 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК [react.dev](#) / [web.dev](#)

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	ПОИСК ПРОБЛЕМ NETWORK Q1 · Q2 · Q3	PROFILER	RENDER vs	С чего начать оптимизацию: меряем, не угадываем.	
ДЕНЬ 2 СРЕДНИЙ	RENDER SURFACE ONTEXT Q4 · Q5 · Q14	RE-RENDER STORM	С	Сужаем поверхность ре-рендеров и боремся с их штормами.	
ДЕНЬ 3 ТЯЖЁЛЫЙ	VIRTUALIZATION Q6 · Q7 · Q8	BIG TABLE		Списки на десятки тысяч строк — без виртуализации никак.	
ДЕНЬ 4 ТЯЖЁЛЫЙ	LIVE TABLE eferredValue Q9 · Q10 · Q11 · Q12	SEARCH	FILTER	used	Динамические данные и тяжёлые вычисления.
ДЕНЬ 5 СРЕДНИЙ	BATCHING NDLE Q13 · Q15 · Q16 · Q17	SELECTORS	DERIVED	BU	Группируем апдейты, точечные подписки, дешёвый derived state.
ДЕНЬ 6 СРЕДНИЙ	DEPENDENCIES Q18 · Q19 · Q20 · Q21 · Q22	CODE SPLITTING	LAZY		Размер бандла, сплиттинг, lazy и его подводные камни.
ДЕНЬ 7 СРЕДНИЙ	SKELETONS ION MEMORY Q23 · Q24 · Q25 · Q26 · Q27 · Q28 · Q29 · Q30	IMAGES BUDGET	FONTS	HYDRAT	UX-детали, ресурсы, утечки и performance budget.

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ

ДЕНЬ 7 – БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

ПОИСК ПРОБЛЕМ PROFILER RENDER vs NETWORK**ЧТО УЧИМ В ЭТОТ ДЕНЬ**

С чего начать оптимизацию: меряем, не угадываем.

ВОПРОСЫ ДНЯ

- Q01 Как найти лишние renders?
- Q02 Как использовать React Profiler?
- Q03 Как понять, что проблема в render, а не в network?

Как найти лишние renders?

ОПИСАНИЕ

Несколько способов: React DevTools Profiler (records рендеры с указанием причины и временем), console.log в render с маркером компонента, Why-Did-You-Render-библиотека (логирует, почему компонент обновился), Highlight Updates в Profiler-tab (мигают перерисованные элементы). Стратегия: профилировать → найти кандидатов → точечная мемоизация.

КАК РАБОТАЕТ

- 01 DevTools → Profiler → Record → действие → анализ.
- 02 Highlight Updates – визуально подсвечивает re-renders.
- 03 Why-Did-You-Render – npm-пакет, патчит React и console.log.
- 04 console.log в начале render – простой и старый приём.
- 05 React.memo + whyDidYouRender = true – селективная диагностика.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Когда дашборд лагает.
- 02 При onboarding – разобрать каскад re-рендеров.
- 03 До любой оптимизации – мерить, не угадывать.

ПОДВОДНЫЕ КАМНИ

- 01 Profiler сам замедляет приложение – не делать выводы по абсолютным числам.
- 02 Why-Did-You-Render – только dev, не в production.
- 03 Преждевременная оптимизация – мерить, не угадывать.
- 04 Лишний рендер !== проблема – проверять реальный impact.

КОД

```
// WDYR setup

// dev only
import wdyr from '@welldone-software/why-did-you-render';
wdyr(React);
Heavy.whyDidYouRender = true;
```

МОГУТ ДОСПРОСИТЬ

- 01 Что показывает React Profiler?
- 02 Чем Profiler в production отличается от dev?
- 03 Когда лишний re-render – не проблема?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/render-and-commit>

Как использовать React Profiler?

ОПИСАНИЕ

Profiler – встроенный инструмент DevTools для записи рендеров. Показывает: какие компоненты перерисовались, сколько заняли (commit duration), почему (props/state/context-change), flame graph и ranked list. Также есть программный `<Profiler>` компонент, чтобы собирать метрики в production через `onRender` callback.

КАК РАБОТАЕТ

- 01 DevTools → Profiler → Record → действие → Stop.
- 02 Flame graph – иерархия с временем.
- 03 Ranked – сортировка по самому медленному.
- 04 Click на компонент – причина рендера.
- 05 Programmatic: `<Profiler id="X" onRender={cb}>`.
- 06 `onRender` (id, phase, actualDuration, baseDuration, startTime, commitTime).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 При жалобах на лаги.
- 02 Для baseline и сравнения после изменений.
- 03 Continuous monitoring через programmatic API.
- 04 A/B тесты – сравнить версии компонента.

ПОДВОДНЫЕ КАМНИ

- 01 Profiler сам замедляет – относительные числа важнее.
- 02 Production-build быстрее dev – измерять production.
- 03 `actualDuration` ≠ `baseDuration` – учитывает мемоизацию.
- 04 Без context – данные бесполезны: нужна сцена.

КОД

// Programmatic

```
<Profiler id="List" onRender={({id, phase, dur}) => {  
  if (dur > 16) sendMetric({ id, phase, dur });  
}}>  
  <List items={items} />  
</Profiler>
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `actualDuration` от `baseDuration` отличается?
- 02 Что такое `commit phase` в Profiler?

03 Когда Profiler обманывает?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/Profiler>

Как понять, что проблема в render, а не в network?

ОПИСАНИЕ

Performance-tab в DevTools показывает все слои: Network, JS, Render, Paint, Composite. Если delay приходится на JS-блок (жёлтый блок Tasks) – render-проблема. Если delay на Network (синий) – сетевая. Часто проблемы смешаны: данные пришли, но тяжёлый рендер дополнительно задерживает. INP высокий → почти всегда render. LCP высокий → часто network.

КАК РАБОТАЕТ

- 01 Performance → Record → действие → анализ frames.
- 02 Network panel – сетевые ожидания.
- 03 Main thread – JS + render + layout + paint.
- 04 INP высокий → JS render.
- 05 LCP высокий → часто image / network / TTFB.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Дебаг "страница медленная".
- 02 До оптимизации – точно понять, что чинить.
- 03 Регрессии перфоманса.

ПОДВОДНЫЕ КАМНИ

- 01 Лагает в dev, fast в prod – не та сцена для замера.
- 02 Network throttling в DevTools может скрывать render-проблемы.
- 03 Слишком короткое окно записи – пропускают long task.

КОД

// Замер по тегам

```
performance.mark('start');
doExpensive();
performance.mark('end');
performance.measure('expensive', 'start', 'end');
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое flame chart?
- 02 Как отличить network-bound от CPU-bound?
- 03 Где смотреть TTFB?

ПОЛНАЯ СТАТЬЯ

<https://developer.chrome.com/docs/devtools/performance>

ДЕНЬ 2 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

RENDER SURFACE**RE-RENDER STORM****CONTEXT****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Сужаем поверхность ре-рендеров и боремся с их штормами.

ВОПРОСЫ ДНЯ

Q04 Как уменьшить render surface?

Q05 Как избежать re-render storm?

Q14 Как оптимизировать context updates?

Как уменьшить render surface?

ОПИСАНИЕ

Render surface – количество компонентов, которые перерисовываются при изменении state. Уменьшить: поднять state ниже (ближе к месту использования), разбить компонент на мелкие, использовать React.memo + стабилизировать props через useMemo/useCallback, вынести изменяющийся state в отдельный поддерево, использовать context-selectors.

КАК РАБОТАЕТ

- 01 State lifting – наоборот: опустить state ниже.
- 02 Composition: `<Parent><Child /></Parent>` – `ChangedChild` не вызовет ререндера sibling.
- 03 React.memo – пропускает рендер при равных props.
- 04 useMemo / useCallback – стабилизируют ссылки.
- 05 Context split – разбить большой Context на узкие.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Дашборды с независимыми виджетами.
- 02 Формы с большим количеством полей.
- 03 Дерево компонентов глубже 5-6 уровней.

ПОДВОДНЫЕ КАМНИ

- 01 Преждевременная мемоизация – code bloat без выгоды.
- 02 React.memo без стабильных props – бесполезен.
- 03 Опускание state ломает API компонента.
- 04 Context split – больше провайдеров.

КОД

// Composition вместо props

```
// плохо – Heavy ререндерится на каждый change в Parent
function Parent({ children, count }) {
  return <div>{count}<Heavy /></div>;
}

// хорошо – Heavy не зависит от count
function Parent({ children, count }) {
  return <div>{count}{children}</div>;
}

<Parent count={n}><Heavy /></Parent>;
```

МОГУТ ДОСПРОСИТЬ

- 01 Что значит "опустить state ниже"?
- 02 Чем composition лучше React.memo?
- 03 Когда мемоизация не нужна?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/sharing-state-between-components>

Как избежать re-render storm?

ОПИСАНИЕ

Re-render storm – когда одно изменение state вызывает каскад рендеров по всему дереву. Причины: большой Context с inline-value, частые setState, глобальный store без селекторов, передача inline-объектов в props. Решения: разбить state на узкие части, селекторы с мемо, batching, useTransition для тяжёлых обновлений.

КАК РАБОТАЕТ

- 01 Batching: несколько setState в одном тике → один render.
- 02 useTransition – пометить апдейт как low-priority.
- 03 Селекторы (useStore(s => s.x)) – подписка на slice.
- 04 useМемо / useCallback стабилизируют props.
- 05 React.мемо – пропуск ре-рендера при equal props.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Real-time дашборды с частыми обновлениями.
- 02 Формы с большим деревом контролов.
- 03 Контекст-провайдеры с часто меняющимся value.

ПОДВОДНЫЕ КАМНИ

- 01 Inline value={{...}} в Context – каждый рендер новый.
- 02 useState на каждое поле формы – много рендеров.
- 03 Subscribe ко всему store без selector – каждый ререндер.
- 04 Без batching – кас ад N setState → N рендеров.

КОД

// useTransition для тяжёлого

```
const [isPending, startTransition] = useTransition();
const onChange = (e) => {
  setQuery(e.target.value); // urgent
  startTransition(() => setFiltered(filter(e.target.value))); // low-priority
};
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем useTransition помогает в storm?
- 02 Когда нужен селектор с shallow?
- 03 Что такое automatic batching в React 18?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useTransition>

Как оптимизировать context updates?

ОПИСАНИЕ

Главная проблема – все consumer'ы Context перерисовываются при изменении value. Решения: разбить большой Context на несколько узких (theme, user, router отдельно); split state и actions в два разных Context (consumer'ы actions не ререндерятся при смене state); useМемо для value (стабильная ссылка); use-context-selector (npm-пакет с подпиской на slice).

КАК РАБОТАЕТ

- 01 Каждый Provider передаёт одно value – все consumer'ы реагируют.
- 02 Object.is сравнивает value – inline {...} = новый каждый раз.
- 03 useМемо: const value = useМемо(() => ({a, b}), [a, b]).
- 04 Split: StateCtx + DispatchCtx – dispatch стабилен.
- 05 use-context-selector – granular подписка.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 auth-state, theme, router – частый кейс.
- 02 Большие провайдеры на верхнем уровне SPA.
- 03 Когда профайлер показывает рендеры от Context.

ПОДВОДНЫЕ КАМНИ

- 01 useМемо с не теми deps – всё равно новая ссылка.
- 02 Слишком много Context-провайдеров → "provider hell".
- 03 use-context-selector – отдельная зависимость, не стандарт.
- 04 React.memo не спасает от Context-рендера.

КОД

// Split state/dispatch

```
const StateCtx = createContext(null);
const DispatchCtx = createContext(null);
function Provider({ children }) {
  const [state, dispatch] = useReducer(r, init);
  return (
    <StateCtx.Provider value={state}>
      <DispatchCtx.Provider value={dispatch}>
        {children}
      </DispatchCtx.Provider>
    </StateCtx.Provider>
  );
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Почему dispatch стабилен?
- 02 Чем use-context-selector отличается от useContext?
- 03 Когда брать Zustand вместо Context?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useContext#optimizing-re-renders-when-passing-objects-and-functions>

VIRTUALIZATION

BIG TABLE

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Списки на десятки тысяч строк – без виртуализации никак.

ВОПРОСЫ ДНЯ

Q06 Когда нужна virtualization?

Q07 Какие edge cases есть у virtualization?

Q08 Как оптимизировать таблицу на 10 000 строк?

ВОПРОС Q06

Когда нужна virtualization?

ОПИСАНИЕ

Виртуализация – рендер только видимой части длинного списка. Нужна, когда элементов сотни / тысячи: даже React.memo не помогает, если в DOM 10 000 элементов. Библиотеки: react-window, react-virtual, TanStack Virtual. Поддерживают fixed-size, variable-size, grid, infinite scroll. Альтернатива – пагинация (если UX позволяет).

КАК РАБОТАЕТ

- 01 Виртуализация рендерит только $N + \text{buffer}$ элементов.
- 02 Высота контейнера + scroll-position → определить видимый диапазон.
- 03 Fixed-size – простой, точный.
- 04 Variable-size – нужен measure-механизм.
- 05 Window / Virtual / TanStack Virtual – основные либы.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Списки 1000+ элементов.
- 02 Чаты, ленты комментариев.
- 03 Большие таблицы.
- 04 Tree-views с большим количеством узлов.

ПОДВОДНЫЕ КАМНИ

- 01 Без виртуализации – DOM огромный, scroll лагает.
- 02 Виртуализация ломает Ctrl+F (find-in-page).
- 03 Accessibility: screen reader не видит off-screen.
- 04 Variable-size – overhead на measure.
- 05 anchor / hash-scroll к скрытому элементу не работает.

КОД

// react-window

```
import { FixedSizeList as List } from 'react-window';
<List height={600} itemCount={items.length} itemSize={48} width="100%">
  {( { index, style } ) => <Row style={style} item={items[index]} />}
</List>
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем react-window от react-virtual отличается?
- 02 Что делать с Ctrl+F?
- 03 Как обеспечить accessibility?

ПОЛНАЯ СТАТЬЯ

<https://react-window.vercel.app/>

ВОПРОС Q07

Какие edge cases есть у virtualization?

ОПИСАНИЕ

Главные сложности: variable-height элементы (нужно measure), accessibility (screen-reader не видит off-screen), Ctrl+F не работает на скрытом, scroll-restoration после mutate, focus-management при удалении видимого элемента, server-side rendering (на сервере нет scroll-position), drag-drop через границы виртуализации.

КАК РАБОТАЕТ

- 01 Variable height: ResizeObserver или measureRef.
- 02 ARIA: aria-rowcount, aria-rowindex для таблиц.
- 03 Scroll restoration: запомнить index + offset, не px.
- 04 On delete: проверить, не находится ли в видимой области.
- 05 SSR: рендерить первые N элементов как fallback.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая нетривиальная виртуализация.
- 02 Когда users жалуются "не нашёл элемент через Ctrl+F".
- 03 Accessibility-аудит.

ПОДВОДНЫЕ КАМНИ

- 01 Element removed from DOM – теряется focus.
- 02 Анимации появления/удаления сложно делать.
- 03 Sticky-header требует доп. логики.
- 04 SEO: контент не в первоначальном HTML.

КОД

// Сохранение scroll по index

```
const listRef = useRef<List>(null);
useEffect(() => {
  if (savedIndex) listRef.current?.scrollToItem(savedIndex);
}, []);
```

МОГУТ ДОСПРОСИТЬ

- 01 Как сделать accessible виртуализацию?
- 02 Как восстановить scroll после refresh?
- 03 Что с drag-drop через границы?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/virtual/latest>

Как оптимизировать таблицу на 10 000 строк?

ОПИСАНИЕ

Стек: TanStack Table (логика) + TanStack Virtual / react-window (виртуализация); fixed-height для простоты; sorting/filtering на стороне сервера для очень больших данных; колоночная виртуализация для широких таблиц; React.memo для строк; стабильные props (useMemo/useCallback); CSS containment.

КАК РАБОТАЕТ

- 01 Виртуализация строк через TanStack Virtual.
- 02 Колоночная виртуализация если колонок 50+.
- 03 Серверная фильтрация/сортировка.
- 04 React.memo Row + key.
- 05 contain: strict / contain: layout – изоляция браузером.
- 06 Pinned columns – отдельный layer.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Админ-панели.
- 02 Финансовые витрины.
- 03 Логи и monitoring-дашборды.
- 04 Catalog tables.

ПОДВОДНЫЕ КАМНИ

- 01 Без виртуализации 10k строк = death.
- 02 Inline-функции в Row = ломает React.memo.
- 03 Sticky-header без оптимизации – янки скролл.
- 04 Без contain – браузер делает layout всей таблицы.

КОД

// Скелет

```
const virtualizer = useVirtualizer({
  count: rows.length,
  getScrollElement: () => parentRef.current,
  estimateSize: () => 40,
  overscan: 5,
});
{virtualizer.getVirtualItems().map(v => <Row key={v.key} />)}
```

МОГУТ ДОСПРОСИТЬ

- 01 Как делать виртуализацию widthwise?

02 Что такое CSS containment?

03 Где делать сортировку – клиент vs сервер?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/table/latest>

LIVE TABLE**SEARCH****FILTER****useDeferredValue****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Динамические данные и тяжёлые вычисления.

ВОПРОСЫ ДНЯ

Q09 Как оптимизировать live-updating table?

Q10 Как оптимизировать search input?

Q11 Как оптимизировать expensive filtering?

Q12 Как использовать useDeferredValue для тяжёлого списка?

ВОПРОС Q09

Как оптимизировать live-updating table?

ОПИСАНИЕ

Live-table – таблица, обновляющаяся на каждый push (биржа, monitoring). Оптимизации: батчить update'ы (собрать N мс – обновить пачкой), обновлять только затронутые строки (по id, не пересоздавать массив), мемо строки (React.мемо по id), useDeferredValue для тяжёлых пересчётов, Suspense + transitions для плавности.

КАК РАБОТАЕТ

- 01 Батчинг апдейтов в store: накопить → flush через setTimeout(50).
- 02 Дифф по id: только изменённые строки → re-рендер.
- 03 React.мемо Row, useCallback handlers.
- 04 useDeferredValue для derived data.
- 05 Виртуализация – обязательна.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Биржевые стаканы.
- 02 Live-логи.
- 03 Прогресс-таблицы (загрузки).
- 04 Multiplayer-доски.

ПОДВОДНЫЕ КАМНИ

- 01 Без батчинга – N апдейтов в секунду = N ререндеров.
- 02 Spread в setState на каждый item – O(N) на каждое.
- 03 Без мемо – все строки re-рендерятся.
- 04 Sort/filter в render – каждый кадр заново.

КОД

// Батч-апдейтер

```
let pending = new Map<string, Row>();
function onTick(row: Row) {
  pending.set(row.id, row);
  if (pending.size === 1) setTimeout(flush, 50);
}
function flush() {
  store.applyBatch([...pending.values()]);
  pending.clear();
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что выбрать – debounce или batching по таймеру?
- 02 Чем canvas быстрее DOM в этом сценарии?
- 03 Как избежать tearing?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useDeferredValue>

ВОПРОС Q10

Как оптимизировать search input?

ОПИСАНИЕ

Search input меняет state на каждое нажатие. Оптимизации: debounce запроса к серверу (~300мс), useDeferredValue для тяжёлого UI, минимальное количество ре-рендеров (state local в компоненте, не в большом store), AbortController для отмены стейлых запросов, мемо результатов.

КАК РАБОТАЕТ

- 01 Debounce: setTimeout 200-400мс перед fetch.
- 02 useDeferredValue: запрос идёт сразу, UI отстаёт.
- 03 AbortController: отменить старый при новом вводе.
- 04 react-query с queryKey включающим q – встроенный кэш.
- 05 isPending от useTransition – индикатор.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой поиск с серверным запросом.
- 02 Локальная фильтрация большого списка.
- 03 Autocomplete / typeahead.

ПОДВОДНЫЕ КАМНИ

- 01 Без debounce – N запросов на N символов.
- 02 Без AbortController – stale-response (см. JS-fundamentals).
- 03 controlled input в большом store – ререндер всего.
- 04 filter в render – N×M operations.

КОД

// react-query + debounce

```
const debounced = useDebounceValue(q, 300);
const { data } = useQuery({
  queryKey: ['search', debounced],
  queryFn: ({ signal }) => api.search(debounced, { signal }),
  enabled: debounced.length > 1,
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем deferred от debounce отличается?
- 02 Когда AbortController обязателен?
- 03 Что такое stale response?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest>

ВОПРОС Q11

Как оптимизировать expensive filtering?

ОПИСАНИЕ

Тяжёлая фильтрация (по огромному массиву): `useMemo` на результат с правильными `deps`, `useDeferredValue` на `input` для отложенного пересчёта, серверная фильтрация при больших наборах, индексирование (`Set` / `Map` для фильтр-критериев), `Web Worker` для CPU-intensive.

КАК РАБОТАЕТ

- 01 `useMemo(() => filter(items, q), [items, q])`.
- 02 `useDeferredValue(q)` – фильтр с low-priority.
- 03 Индекс: `Set<string>` для $O(1)$ lookup.
- 04 `Web Worker` – для огромных датасетов.
- 05 Виртуализация результата.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Локальные таблицы / списки.
- 02 Поиск без сервера (offline-first).
- 03 Большие справочники.

ПОДВОДНЫЕ КАМНИ

- 01 `useMemo` без правильных `deps` – кэш не работает.
- 02 `Filter` в каждом `render` без `useMemo` – $O(N)$ на каждое.
- 03 `Worker overhead` для маленьких массивов.
- 04 `Index` создаётся в `render` – дороже чем сам `filter`.

КОД

```
// useDeferredValue + useMemo
```

```
const deferred = useDeferredValue(q);
const filtered = useMemo(() => filter(items, deferred), [items, deferred]);
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда вынести в `Web Worker`?
- 02 Чем индекс быстрее `filter`?
- 03 Что такое `FlexSearch`?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useDeferredValue>

Как использовать `useDeferredValue` для тяжёлого списка?

ОПИСАНИЕ

`useDeferredValue(value)` возвращает «отложенную» версию `value`. Когда исходное `value` меняется, `deferred` сначала остаётся старым, потом «догоняет» в `low-priority transition`. UI при этом остаётся отзывчивым: ввод не лагает, тяжёлый список обновляется чуть позже. Можно показать `isPending (deferred !== value)` для индикатора.

КАК РАБОТАЕТ

- 01 `const deferred = useDeferredValue(q).`
- 02 Сначала `deferred = старое`; потом — новое (через `transition`).
- 03 `deferred !== q` → "в процессе".
- 04 Можно использовать для тяжёлого `render-фильтра`.
- 05 Не уменьшает CPU — переставляет приоритеты.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Тяжёлый локальный фильтр / поиск.
- 02 Чарт, перерисовывающийся при изменении `props`.
- 03 Список 1000+ строк, фильтруемый онлайн.

ПОДВОДНЫЕ КАМНИ

- 01 Не помогает с маленькими массивами — `overhead`.
- 02 `deferred`-объект — сравнение по ссылке (`Object.is`).
- 03 Не подходит для `async`-данных (не `fetch`-кэш).
- 04 `isPending` с переходами анимаций может мигать.

КОД

// Stale-индикатор

```
function HeavyList({ filter }: { filter: string }) {
  const deferred = useDeferredValue(filter);
  const isStale = deferred !== filter;
  return (
    <div style={{ opacity: isStale ? 0.5 : 1 }}>
      <List items={filterFn(deferred)} />
    </div>
  );
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `deferred` от `useTransition` отличается?

02 Что показать пользователю во время stale?

03 Когда useDeferredValue лишний?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useDeferredValue>

BATCHING**SELECTORS****DERIVED****BUNDLE****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Группируем апдейты, точечные подписки, дешёвый derived state.

ВОПРОСЫ ДНЯ

Q13 Как использовать batching для real-time updates?

Q15 Как оптимизировать selector-based subscriptions?

Q16 Как избежать expensive derived calculations?

Q17 Как оптимизировать bundle size?

ВОПРОС Q13

Как использовать batching для real-time updates?

ОПИСАНИЕ

Real-time обновления (WebSocket-сообщения, stream-data) приходят пакетами или часто. Без батчинга – каждое сообщение → setState → render. С батчингом – собираем 50-100мс пачку и применяем одним setState. React 18 батчит автоматически, но для high-frequency сценариев нужен application-level батчинг.

КАК РАБОТАЕТ

- 01 React 18 automatic batching: несколько setState в одном тике → один render.
- 02 Application-level: накапливать в Map / Array, flush через setTimeout / rAF.
- 03 Throttle: применять не чаще N мс.
- 04 rAF-based батчинг для синхронности с paint.
- 05 Selective: батчить только high-frequency, urgent – сразу.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Биржевые тики.
- 02 Чат-сообщения в высоконагруженном канале.
- 03 Multiplayer-кроссхаиры.
- 04 Real-time графики.

ПОДВОДНЫЕ КАМНИ

- 01 Слишком большое окно – задержка отображения.
- 02 Слишком маленькое – нет выгоды.
- 03 Race с reconnect – потерять накопленное.
- 04 Батч с приоритетами требует custom logic.

КОД

// rAF-батчинг

```
let pending: Update[] = [];  
let scheduled = false;  
function onMessage(u: Update) {  
  pending.push(u);  
  if (!scheduled) {  
    scheduled = true;  
    requestAnimationFrame(() => {  
      store.applyBatch(pending);  
      pending = [];  
      scheduled = false;  
    });  
  }  
}
```

```
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем automatic batching отличается от ручного?
- 02 Когда rAF лучше setTimeout?
- 03 Что такое flushSync?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useTransition>

ВОПРОС Q15

Как оптимизировать selector-based subscriptions?

ОПИСАНИЕ

Селекторы в Zustand / Redux / use-context-selector – подписка на конкретный slice store. Оптимизации: shallow-сравнение для объектных селекторов, createSelector (reselect) для мемоизации, минимальные селекторы (один primitive – ререндер только при его смене), не выбирать весь объект если нужно одно поле.

КАК РАБОТАЕТ

- 01 Zustand: `useStore(s => s.x)` – подписка на `x`.
- 02 Object selector с `shallow`: `useStore(s => ({ a, b }), shallow)`.
- 03 `reselect` (Redux): `createSelector` с input/output.
- 04 Selector должен быть `pure` и стабильным.
- 05 Сравнение по умолчанию – `Object.is`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой store с несколькими полями.
- 02 Большие state-trees (Redux dashboards).
- 03 Дашборды с много виджетов.

ПОДВОДНЫЕ КАМНИ

- 01 Selector объекта без `shallow` – ререндер на каждое.
- 02 Селектор-фабрика в `render` – пересоздаётся.
- 03 Тяжёлый селектор без мемо – пересчитывается.
- 04 `filter` / `map` внутри селектора без `reselect` – новый массив каждый раз.

КОД

```
// Zustand shallow
```

```
import { shallow } from 'zustand/shallow';
const { a, b } = useStore(s => ({ a: s.a, b: s.b }), shallow);
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое `shallow` vs `deep`?
- 02 Чем `reselect` помогает?
- 03 Что такое `stable selector`?

ПОЛНАЯ СТАТЬЯ

<https://docs.pmnd.rs/zustand/guides/typescript>

Как избежать expensive derived calculations?

ОПИСАНИЕ

Derived state – вычисленные значения. Если расчёт тяжёлый, оборачивать в useМемо с правильными deps. Для глобального state – мемоизированные селекторы (reselect, createSelector). Не хранить derived в state (рассинхрон), а вычислять на лету. Если derived нужен в нескольких местах – поднять в selector, не в каждом компоненте.

КАК РАБОТАЕТ

- 01 useМемо(() => calc(items), [items]).
- 02 createSelector – кэш по ссылке аргументов.
- 03 MobX computed – автоматическая мемоизация.
- 04 Не хранить derived в state.
- 05 Иногда дешевле пересчитать, чем мемоизировать (маленькие данные).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Total / sum / average больших списков.
- 02 Filter / sort / group крупных коллекций.
- 03 Агрегация данных для charts.
- 04 View-models из raw-data.

ПОДВОДНЫЕ КАМНИ

- 01 useМемо с неправильными deps – кэш не работает.
- 02 Слишком мелкие useМемо – overhead больше выгоды.
- 03 Хранение derived в state → рассинхрон.
- 04 createSelector с нестабильными input → пересчёт.

КОД

```
// reselect
```

```
const selectVisibleTodos = createSelector(
  [(s: State) => s.todos, (s: State) => s.filter],
  (todos, filter) => todos.filter(t => match(t, filter))
);
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда useМемо – overhead?
 - 02 Чем reselect помогает?
 - 03 Что такое structural sharing?
-

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useMemo>

Как оптимизировать bundle size?

ОПИСАНИЕ

Меры: tree-shaking (импортировать только нужное), code-splitting по роутам и компонентам, замена тяжёлых зависимостей (lodash → lodash-es, moment → date-fns/dayjs), gzip/brotli, dynamic imports, анализ через bundle-analyzer (webpack-bundle-analyzer, Rollup-visualizer), удаление dead-code, использование modern-build (esnext).

КАК РАБОТАЕТ

- 01 Tree-shaking: импорт по имени, ESM-модули.
- 02 Code-splitting: React.lazy + Suspense.
- 03 Dynamic import('heavy-lib').
- 04 Bundle analyzer: визуализация sizes.
- 05 Modern build: target=esnext + legacy fallback.
- 06 Compression: brotli > gzip.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любое production-приложение.
- 02 Аудит после долгой разработки.
- 03 Внедрение новой большой зависимости.

ПОДВОДНЫЕ КАМНИ

- 01 import _ from 'lodash' тащит всё.
- 02 Heavy-deps в shared chunk → загружается всеми.
- 03 Без analysis не знаем, что весит.
- 04 Слишком много мелких chunks → много network requests.

КОД

// Импорт по имени

```
// плохо
import _ from 'lodash';
// хорошо
import debounce from 'lodash/debounce';
// или lodash-es
import { debounce } from 'lodash-es';
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое tree-shaking?
- 02 Чем dynamic import от React.lazy отличается?

03 Какие альтернативы lodash?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/reduce-javascript-payloads-with-code-splitting>

DEPENDENCIES**CODE SPLITTING****LAZY****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Размер бандла, сплиттинг, lazy и его подводные камни.

ВОПРОСЫ ДНЯ

- Q18 Как найти тяжёлые dependencies?
- Q19 Что такое code splitting?
- Q20 Что такое route-based splitting?
- Q21 Что такое component-level lazy loading?
- Q22 Когда lazy loading ухудшает UX?

ВОПРОС Q18

Как найти тяжёлые dependencies?

ОПИСАНИЕ

Инструменты: webpack-bundle-analyzer / rollup-plugin-visualizer (визуальная карта bundle), source-map-explorer (анализ по source-maps), bundlephobia.com (size конкретного npm-пакета), size-limit (CI-проверка). Стратегия: проверить каждую новую зависимость перед добавлением, регулярно аудитить.

КАК РАБОТАЕТ

- 01 webpack-bundle-analyzer – после build → HTML с размерами.
- 02 source-map-explorer – анализ по source-maps.
- 03 size-limit – CI-чек на growth.
- 04 bundlephobia.com – UI для проверки npm-пакетов.
- 05 dercheck – найти неиспользуемые.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Каждая новая dependency.
- 02 Регулярные аудиты bundle.
- 03 Перед production-релизом.
- 04 При жалобах на медленную загрузку.

ПОДВОДНЫЕ КАМНИ

- 01 Без source-maps анализ неточный.
- 02 Tree-shaking не работает для CommonJS-модулей.
- 03 Дубли версий в monorepo – больше bundle.
- 04 Polyfills для legacy = +KB.

КОД

```
// size-limit конфиг
{
  // .size-limit.json
  [
    { "path": "dist/main.js", "limit": "100 KB" }
  ]
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое source-map-explorer?
- 02 Чем bundlephobia полезен?
- 03 Как проверять PR на bundle-growth?

ПОЛНАЯ СТАТЬЯ

<https://github.com/webpack-contrib/webpack-bundle-analyzer>

ВОПРОС Q19

Что такое code splitting?

ОПИСАНИЕ

Code splitting – разбиение JS-bundle на отдельные файлы (chunks), которые загружаются по необходимости. Уменьшает initial bundle (быстрый старт), пользователь грузит только то, что использует. Реализуется через `dynamic import('./heavy.js')` или `React.lazy`. Bundlers (webpack, vite, rollup) автоматически создают chunks.

КАК РАБОТАЕТ

- 01 `Dynamic import` → bundler создаёт отдельный chunk.
- 02 Chunk загружается при первом `import`.
- 03 `React.lazy(() => import(...)) + Suspense`.
- 04 Manual chunks: vendor, common.
- 05 `Preload / prefetch` для предзагрузки.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Routes – разные страницы как отдельные chunks.
- 02 Компоненты, не нужные сразу (modals, settings).
- 03 Тяжёлые библиотеки (charting, editor, video).
- 04 `i18n` – словари по языкам.

ПОДВОДНЫЕ КАМНИ

- 01 Слишком мелкое разбиение – много network requests.
- 02 Без `Suspense fallback` – UI сломается.
- 03 `Chunk-load error` – нужен `retry / fallback`.
- 04 `Vendor-chunk` обновляется при любом `upgrade` – выбор стратегии важен.

КОД

```
// Dynamic import
const Chart = React.lazy(() => import('./Chart'));
<Suspense fallback={<Spinner />}>
  <Chart />
</Suspense>
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `dynamic import` от `React.lazy` отличается?
- 02 Что такое `chunk-load error`?
- 03 Как преконсчёт chunks?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/lazy>

ВОПРОС Q20

Что такое route-based splitting?

ОПИСАНИЕ

Самый частый и эффективный вид code splitting: каждая страница (роут) – отдельный chunk. Пользователь грузит только initial route + общие вендор-chunks. Все современные роутеры (React Router, Next.js, TanStack Router) поддерживают через lazy-routes из коробки.

КАК РАБОТАЕТ

- 01 React Router: `route element={lazy(() => import(...))}`.
- 02 Next.js Pages – splitting автоматическое.
- 03 Next App Router – RSC + client-component split.
- 04 TanStack Router – типобезопасный lazy-loader.
- 05 Preload на hover/focus – UX-улучшение.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любое многостраничное приложение.
- 02 SPA с десятками роутов.
- 03 Микро-фронтенды.

ПОДВОДНЫЕ КАМНИ

- 01 При первом visit на страницу – задержка загрузки.
- 02 Без preload – нажал → ждёшь.
- 03 Vendor-chunks могут дублироваться.
- 04 Chunk-error при offline – UI ломается.

КОД

// React Router

```
const Settings = lazy(() => import('./pages/Settings'));  
<Routes>  
  <Route path="/settings" element={  
    <Suspense fallback={<Spinner />}><Settings /></Suspense>  
  } />  
</Routes>
```

МОГУТ ДОСПРОСИТЬ

- 01 Как preload route на hover?
- 02 Что такое splitChunks в webpack?
- 03 Чем Next.js auto-splitting отличается?

ПОЛНАЯ СТАТЬЯ

<https://reactrouter.com/en/main/route/lazy>

ВОПРОС Q 21

Что такое component-level lazy loading?

ОПИСАНИЕ

Загрузка отдельного компонента по требованию (не целой страницы). Полезно для тяжёлых компонентов внутри страницы, которые видны не сразу: модальки, редакторы, графики, рассчитанные диаграммы. Технически – `React.lazy` + `Suspense` вокруг компонента.

КАК РАБОТАЕТ

- 01 `React.lazy(() => import('./Heavy'))`.
- 02 `Suspense fallback={<Skeleton />}`.
- 03 Загружается при первом mount компонента.
- 04 Можно `preload` на `hover` ссылки/кнопки.
- 05 `Sentry` / `Bugsnag` для отслеживания `chunk-errors`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Тяжёлые модальки.
- 02 WYSIWYG-редакторы.
- 03 Чарты / 3D-графика.
- 04 Видеоплееры.
- 05 Rich-text input.

ПОДВОДНЫЕ КАМНИ

- 01 `Suspense` без `fallback` – пустота.
- 02 Lazy-component внутри мемо может терять `Suspense-boundary`.
- 03 `Chunk-error` при чисто-online сценарии – нужен `fallback`.
- 04 SSR + lazy – нюансы (`suspenseFallbackEnabled`).

КОД

// Lazy modal

```
const SettingsModal = React.lazy(() => import('./SettingsModal'));
{open && (
  <Suspense fallback={<Spinner />}>
    <SettingsModal onClose={close} />
  </Suspense>
)}
```

МОГУТ ДОСПРОСИТЬ

- 01 Как `preload` компонент?
- 02 Что такое `chunk-error`?

03 Как обработать ошибку lazy-load?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/lazy>

ВОПРОС Q22

Когда lazy loading ухудшает UX?

ОПИСАНИЕ

Когда задержка загрузки замечает пользователь: критичные элементы (header, navigation), часто-используемые модалки, элементы above-the-fold. Также lazy для очень маленьких компонентов – overhead сетевого запроса больше выгоды. И при плохом fallback (просто spinner на пустом месте) – UX мерцает.

КАК РАБОТАЕТ

- 01 Сетевой round-trip ~50-200мс на chunk-load.
- 02 Cold cache – больше времени.
- 03 Suspense fallback – мигание UI.
- 04 На медленной сети lazy вреден.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Критичные элементы – НЕ lazy.
- 02 Часто видимые компоненты – НЕ lazy.
- 03 Маленькие (<5KB) компоненты – НЕ lazy.
- 04 Compound-компоненты, где части идут вместе – общий chunk.

ПОДВОДНЫЕ КАМНИ

- 01 Lazy main navigation → все страницы ждут header.
- 02 Lazy частая модалка → каждое открытие – задержка.
- 03 Скелетон не похож на финальный UI – disorienting.
- 04 Skeleton без min-display-time мигает.

КОД

// Preload на hover

```
const Heavy = lazy(() => import('./Heavy'));  
<button  
  onMouseEnter={() => import('./Heavy')} // preload  
  onClick={() => setOpen(true)}  
>Open</button>
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда лучше eager-load?
- 02 Что такое min-display-time для skeleton?
- 03 Чем prefetch от preload отличается?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/code-splitting-suspense>

ДЕНЬ 7

НАГРУЗКА

СРЕДНИЙ

8 ВОПРОСОВ

SKELETONS**IMAGES** **FONTS****HYDRATION****MEMORY****BU****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

UX-детали, ресурсы, утечки и performance budget.

ВОПРОСЫ ДНЯ

- Q23 Как проектировать skeleton/loading state?
- Q24 Как оптимизировать images?
- Q25 Как оптимизировать fonts?
- Q26 Как оптимизировать hydration?
- Q27 Как работать с memory leaks?
- Q28 Как найти detached DOM nodes?
- Q29 Как профилировать memory?
- Q30 Как поставить performance budget?

Как проектировать skeleton/loading state?

ОПИСАНИЕ

Skeleton – пустые блоки в форме финального UI, показываются вместо данных пока они грузятся. Принципы: совпадает с финальным layout (нет сдвига → лучше CLS), сдержанная анимация (shimmer / pulse), быстро исчезает (min-display-time, чтобы не мигало). Альтернатива – Suspense + fallback в react-query.

КАК РАБОТАЕТ

- 01 Skeleton match'ит финальный layout по размерам.
- 02 CSS pulse / shimmer.
- 03 min-display-time 200мс – чтобы избежать мерцания.
- 04 Виды: list, card, table, чарт – разные шаблоны.
- 05 Suspense fallback в react-query / Next.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Initial load страницы.
- 02 Refetch с pending state.
- 03 Lazy-loaded компонент.
- 04 После navigate с suspense.

ПОДВОДНЫЕ КАМНИ

- 01 Skeleton не совпадает с финалом → CLS прыгает.
- 02 Без min-display-time – мигает.
- 03 Слишком пёстрая анимация – раздражает.
- 04 Skeleton на быстрой сети показывается 50мс – хуже, чем без.

КОД

// min-display-time

```
function useDelayedLoading(loading: boolean, min = 200) {
  const [show, setShow] = useState(loading);
  useEffect(() => {
    if (!loading) {
      const t = setTimeout(() => setShow(false), min);
      return () => clearTimeout(t);
    }
    setShow(true);
  }, [loading]);
  return show;
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем skeleton от spinner лучше?
- 02 Зачем min-display-time?
- 03 Как избежать CLS?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/cls>

Как оптимизировать images?

ОПИСАНИЕ

Главное: правильный формат (WebP / AVIF), responsive (<picture>, srcset), lazy (loading="lazy"), размеры в HTML (для CLS), priority hints (fetchpriority="high" для hero), CDN с автоконверсией. В Next.js – встроенный <Image>, который делает большую часть.

КАК РАБОТАЕТ

- 01 Формат: AVIF (новый) > WebP > JPG/PNG.
- 02 <picture> + <source> для нескольких форматов.
- 03 srcset + sizes для responsive.
- 04 loading="lazy" для off-screen.
- 05 decoding="async".
- 06 fetchpriority="high" для LCP-картинки.
- 07 width / height – обязательно (CLS).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любые статичные изображения.
- 02 Hero / above-the-fold (приоритет).
- 03 Галереи, ленты, карточки.
- 04 User-generated content.

ПОДВОДНЫЕ КАМНИ

- 01 PNG для фото – огромный размер.
- 02 Без width/height – CLS прыгает.
- 03 Lazy для hero – позднее LCP.
- 04 fetchpriority="high" на всех – теряет смысл.

КОД

// Полный набор

```
<picture>
  <source srcSet="img.avif" type="image/avif" />
  <source srcSet="img.webp" type="image/webp" />
  
</picture>
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем AVIF от WebP отличается?

02 Что такое fetchpriority?

03 Чем Next/Image удобен?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/learn/images>

Как оптимизировать fonts?

ОПИСАНИЕ

Шрифты часто блокируют рендер. Меры: `font-display: swap` (показать fallback сразу), `preload` критичных шрифтов, `subset` (только используемые символы), `self-host` вместо Google Fonts CDN, `variable fonts` (один файл вместо N вариантов), `unicode-range` для оптимизации.

КАК РАБОТАЕТ

- 01 `@font-face { font-display: swap; }` – сразу fallback.
- 02 `<link rel="preload" as="font" type="font/woff2" crossorigin>`.
- 03 Subset: `glyph-set` по используемым символам.
- 04 Variable fonts – единый файл с разными `weight/italic`.
- 05 Self-host: меньше DNS lookup, больше control.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой кастомный шрифт.
- 02 Сайты с критичной типографикой.
- 03 Мобильные / slow-3G сценарии.

ПОДВОДНЫЕ КАМНИ

- 01 Без `font-display` – FOIT (текст невидим до загрузки).
- 02 Без `subset` – лишние символы.
- 03 Без `preload` – поздняя загрузка.
- 04 Google Fonts CDN – лишний DNS / TLS.
- 05 Несовместимый fallback по метрикам – CLS.

КОД

```
// @font-face + preload
```

```
/* CSS */
@font-face {
  font-family: 'Inter';
  src: url('/fonts/inter.woff2') format('woff2');
  font-display: swap;
}
<!-- HTML -->
<link rel="preload" href="/fonts/inter.woff2" as="font" type="font/woff2" crossorigin>
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `font-display: swap` от `optional` отличается?

- 02 Что такое subset?
- 03 Чем variable font выгоден?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/font-best-practices>

Как оптимизировать hydration?

ОПИСАНИЕ

Hydration – присоединение JS к серверному HTML. Может быть дорогим: всё дерево вызывается с props. Оптимизации: streaming SSR (поток HTML по частям), selective/partial hydration (гидрация только нужных островков), React Server Components (часть кода вообще не идёт на клиент), lazy hydration для off-screen. В Next App Router многое автоматически.

КАК РАБОТАЕТ

- 01 Streaming SSR – отдача HTML кусками.
- 02 Selective hydration – гидрация по приоритету.
- 03 RSC (React Server Components) – server-only.
- 04 Hydration starts при page-load, blocks main thread.
- 05 Suspense boundaries позволяют частичную гидрацию.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 SSR-приложения с большим деревом.
- 02 Next App Router.
- 03 Remix.
- 04 Микро-фронтенды.

ПОДВОДНЫЕ КАМНИ

- 01 Hydration mismatch – warning + повторный рендер.
- 02 Большой client-bundle = долгая гидрация.
- 03 RSC ограничивает использование state/effects.
- 04 Astro / Qwik – другие подходы (resumability).

КОД

// Suspense для частичной гидрации

```
<Suspense fallback={<Spinner />}>
  <HeavyClientWidget />
</Suspense>
<!-- остальное гидрируется параллельно -->
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое streaming SSR?
- 02 Чем RSC от обычного компонента отличается?
- 03 Что такое resumability?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react-dom/client/hydrateRoot>

Как работать с memory leaks?

ОПИСАНИЕ

Memory leak – объекты, которые не освобождаются GC, хотя по логике уже не нужны. В React-приложениях частые источники: подписки без cleanup, обработчики событий без remove, замыкания над большими данными в долгоживущих коллбэках, детачированные DOM-узлы (см. Q28). Профилирование через DevTools Memory – снапшоты + сравнение.

КАК РАБОТАЕТ

- 01 Снапшот → действие → снапшот → diff.
- 02 Retainers – что держит объект.
- 03 Listener / observer / interval без cleanup.
- 04 Detached DOM nodes – типичный сигнал.
- 05 Memory growth по сессии – линейный = leak.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Долгоживущие SPA, у которых растёт RAM.
- 02 Регрессии после релиза.
- 03 Перед production-релизом.

ПОДВОДНЫЕ КАМНИ

- 01 DevTools Memory сам потребляет → закрыть Console.
- 02 Каждое действие даёт небольшой grow – нормально.
- 03 Утечки видны только под нагрузкой.
- 04 В dev StrictMode эффекты двойные – лишний шум.

КОД

// Cleanup в useEffect

```
useEffect(() => {  
  const t = setInterval(tick, 1000);  
  return () => clearInterval(t);  
}, []);
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие частые источники leaks в React?
- 02 Как читать retainer chain?
- 03 Что такое detached DOM?

ПОЛНАЯ СТАТЬЯ

<https://developer.chrome.com/docs/devtools/memory-problems>

ВОПРОС Q28

Как найти detached DOM nodes?

ОПИСАНИЕ

Detached DOM – узлы, удалённые из дерева, но удерживаемые JS-ссылкой (в массиве, в state, в замыкании). DevTools Memory → Heap snapshot → поиск по "Detached". Покажет HTMLElement не в document. Дальше – retainer chain показывает, кто держит.

КАК РАБОТАЕТ

- 01 Heap snapshot → Filter → "Detached".
- 02 Retainer panel – путь до root.
- 03 Часто: ref сохранён в массиве, не очищается.
- 04 Очистить: ref = null после unmount.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Дебаг roughly утечек.
- 02 После большого refactor.
- 03 Жалобы пользователей на тормоза long-session.

ПОДВОДНЫЕ КАМНИ

- 01 DevTools удерживает свои ссылки на DOM – закрыть.
- 02 В Strict mode dev отображает дубли.
- 03 Не все detached = leak (могут быть skip кадры).

КОД

// Очистка ref

```
useEffect(() => {  
  const node = document.createElement('div');  
  refs.add(node);  
  return () => { refs.delete(node); };  
}, []);
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое retainer?
- 02 Чем help retainer chain?
- 03 Какие типичные ошибки приводят к detached?

ПОЛНАЯ СТАТЬЯ

<https://developer.chrome.com/docs/devtools/memory-problems/heap-snapshots>

Как профилировать memory?

ОПИСАНИЕ

Инструменты: Chrome DevTools Memory tab (Heap snapshot, Allocation timeline, Allocation sampling), Performance API (performance.memory.* – Chrome only), measureUserAgentSpecificMemory() (стандартный API, требует cross-origin isolation). В Node – heapdump, --inspect, clinic.js.

КАК РАБОТАЕТ

- 01 Heap snapshot – детальная фотография в момент времени.
- 02 Allocation timeline – что аллоцировалось во время записи.
- 03 Allocation sampling – лёгкий sampling-профайлер.
- 04 performance.memory.usedJSHeapSize – Chrome only.
- 05 measureUserAgentSpecificMemory – точная цифра, новый стандарт.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любые подозрения на утечки.
- 02 Перед production release.
- 03 Регрессии после релизов.
- 04 Long-session мониторинг.

ПОДВОДНЫЕ КАМНИ

- 01 performance.memory неточный.
- 02 Snapshot большой → долго грузится.
- 03 Allocation timeline replay не сохраняет state.
- 04 DevTools удерживает что попало.

КОД

```
// measureUserAgentSpecificMemory
if (performance.measureUserAgentSpecificMemory) {
  const r = await performance.measureUserAgentSpecificMemory();
  console.log(r.bytes);
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем allocation timeline от snapshot отличается?
 - 02 Что такое cross-origin isolation?
 - 03 Как мерить memory в Node?
-

ПОЛНАЯ СТАТЬЯ

<https://developer.chrome.com/docs/devtools/memory-problems>

Как поставить performance budget?

ОПИСАНИЕ

Performance budget – числовые ограничения на метрики и ресурсы: bundle size $\leq$ 200KB, LCP $\leq$ 2.5с, INP $\leq$ 200мс, image-size $\leq$ 100KB. Реализуется через CI-чеки (Lighthouse CI, size-limit, bundlwatch) и dashboards (RUM alerts). Бюджет даёт команде ясный target и защищает от деградации.

КАК РАБОТАЕТ

- 01 Define metrics + thresholds: LCP, INP, CLS, JS bundle, CSS bundle, images.
- 02 CI-чек: build → измерить → fail если над бюджетом.
- 03 Lighthouse CI: lhci CLI в pipeline.
- 04 size-limit: per-bundle limits.
- 05 RUM-alerts: triggered при ухудшении прода.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Команды с performance-целями.
- 02 Production-приложения.
- 03 После регрессий – задание границ.
- 04 Multi-team проекты – дисциплина.

ПОДВОДНЫЕ КАМНИ

- 01 Бюджет без owner – игнорируется.
- 02 Слишком жёсткий – блокирует фичи.
- 03 Без RUM-валидации lab может расходиться с реальностью.
- 04 Бюджет не пересматривается с ростом приложения.

КОД

```
// size-limit + Lighthouse CI
```

```
// .size-limit.json
[{"path": "dist/main.js", "limit": "100 KB" }]

// lighthouseci.json
{
  "ci": {
    "assert": {
      "assertions": {
        "largest-contentful-paint": ["error", { "maxNumericValue": 2500 }]
      }
    }
  }
}
```

```
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие метрики в бюджете?
- 02 Кто owner бюджета в команде?
- 03 Как корректировать бюджет?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/performance-budgets-101>

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ [] Описать, как искать лишние renders и пользоваться React Profiler (Q1, Q2, Q3)
- ☐ [] Сказать, как уменьшить render surface и побеждать re-render storm и Context-проблемы (Q4, Q5, Q14)
- ☐ [] Описать virtualization для больших таблиц + edge-cases (Q6, Q7, Q8)
- ☐ [] Расписать оптимизации live-table, search input, expensive filtering, useDeferredValue (Q9-Q12)
- ☐ [] Объяснить batching real-time updates, selectors, derived state, bundle size (Q13, Q15, Q16, Q17)
- ☐ [] Сказать, как находить тяжёлые dependencies и применять code-splitting / lazy (Q18-Q22)
- ☐ [] Описать skeletons, image / font оптимизации, hydration (Q23-Q26)
- ☐ [] Описать поиск memory leaks, detached DOM и memory profiling (Q27-Q29)
- ☐ [] Сформулировать performance budget и его CI-чеки (Q30)

EVENT LOOP / ASYNC МЕТОДИЧКА v1

МАТЕРИАЛ ОПИРАЕТСЯ НА learn.javascript.ru